

SW-I

SISTEMAS WEB I

Prof. Anderson Vanin

AULA 09 - Funções em PHP

1. O que é uma Função e por que precisamos dela?

Imagine que você trabalha em uma loja e, toda vez que um cliente faz uma compra, você precisa calcular o valor do frete, o imposto e o desconto, e depois imprimir um recibo. Se você tiver que escrever todas as fórmulas matemáticas para cada cliente, seu dia será cansativo e as chances de errar um cálculo no meio do caminho serão imensas!

1. O que é uma Função e por que precisamos dela?

Na programação, nós resolvemos isso com Funções.

Uma função é como uma máquina com uma tarefa específica. Pense em um liquidificador:

1. Você coloca os ingredientes (Entrada de dados / Parâmetros).
2. Ele processa tudo lá dentro (Lógica / Código).
3. Ele te entrega uma vitamina pronta (Saída de dados / Retorno).

1. O que é uma Função e por que precisamos dela?

Os 3 Superpoderes das Funções:

- **Reutilização:** Escreva o código uma vez e use quantas vezes quiser.
- **Organização:** Seu arquivo principal não fica com milhares de linhas bagunçadas.
- **Manutenção fácil:** Se a regra do imposto mudar, você altera apenas dentro da função, e o sistema inteiro é atualizado automaticamente.

2. Criando a Função Mais Simples (Sem parâmetros e sem retorno)

Para criar uma função em PHP, usamos a palavra-chave **function**, damos um nome a ela e abrimos e fechamos chaves { } para guardar o nosso código.

```
<?php
// 1. Declarando (criando) a função
function exibirMensagemPadrao() {
    echo "<p>Obrigado por comprar em nossa loja! Volte sempre.</p>";
}

// 2. Chamando (usando) a função
exibirMensagemPadrao();
exibirMensagemPadrao(); // Posso chamar quantas vezes quiser!
?>
```

3. Funções com Parâmetros (Enviando dados para a máquina)

Nossa função anterior faz sempre a mesma coisa. Mas e se quisermos que ela seja dinâmica? Nós usamos os parâmetros, que são variáveis que colocamos dentro dos parênteses.

Vamos criar uma função que diz "Olá" de forma personalizada.

```
<?php
```

```
// A variável $nome é o nosso parâmetro (o ingrediente)
```

```
function saudarCliente($nome) {
```

```
    echo "<p>Olá, " . $nome . "! Bem-vindo à nossa loja.</p>";
```

```
}
```

```
// Quando chamamos a função, entregamos o valor que ela pediu
```

```
saudarCliente("Ana");
```

```
saudarCliente("Carlos");
```

```
?>
```

4. Funções com Retorno (A vitamina pronta)

Até agora, nossas funções apenas imprimiram textos na tela (echo). Mas, na vida real, geralmente queremos que uma função faça um cálculo complexo e nos devolva o resultado para usarmos em outro lugar. Para isso, usamos a palavra `return`. Vamos criar uma função que calcula um desconto de 10%.

4. Funções com Retorno (A vitamina pronta)

```
<?php
function calcularDesconto($valorDaCompra) {
    // Calculamos o desconto (10% do valor)
    $desconto = $valorDaCompra * 0.10;

    // Subtraímos o desconto do valor original
    $valorFinal = $valorDaCompra - $desconto;

    // Devolvemos o valor final pronto!
    return $valorFinal;
}

// Usando a função na prática:
$totalCarrinho = 250.00;
$valorA Pagar = calcularDesconto($totalCarrinho);

echo "Valor original: R$ " . $totalCarrinho . "<br>";
echo "Valor com desconto: R$ " . $valorAPagar;
?>
```


5. Exemplo Prático Completo: Carrinho de Compras

Vamos juntar tudo em um cenário real. Uma função que recebe o preço de um produto e a quantidade comprada, e nos diz quanto custará no total, já aplicando regras.

5. Exemplo Prático Completo: Carrinho de Compras

```
<?php

// Função que recebe DOIS parâmetros

function calcularTotal($precoUnitario, $quantidade) {
    $total = $precoUnitario * $quantidade;
    // Se comprou mais de 5 itens,
    // ganha frete grátis (desconto extra de R$20)
    if ($quantidade > 5) {
        $total = $total - 20;
    }
    return $total;
}

// Usando no sistema da loja:

$precoMouse = 50.00;

$qtdComprada = 6;

$valorFinalDaVenda = calcularTotal($precoMouse,
    $qtdComprada);

echo "Você comprou " . $qtdComprada . " mouses.<br>";
echo "O total a pagar é: R$ " . $valorFinalDaVenda;

?>
```

6. Lista de Exercícios: Funções em PHP

Exercício 1: A Máquina Simples (Sem parâmetros e sem retorno). Crie uma função chamada **acionarCenaNoturna()**. Esta função não precisa receber nenhum parâmetro. O objetivo dela é apenas imprimir na tela a mensagem: *"Luzes Zigbee apagadas, persianas fechadas e alarme ativado."*. Em seguida, chame a função no seu código para ver a mensagem aparecer.

6. Lista de Exercícios: Funções em PHP

Exercício 2: Enviando um Comando (Com 1 parâmetro). Imagine que estamos programando o painel de controle de um braço robótico. Crie uma função chamada **movimentarMotor(\$graus)** que receba um número como parâmetro. A função deve imprimir na tela a mensagem: "*Movendo o servo motor para [graus] graus.*". Teste a função passando o valor **90**.

6. Lista de Exercícios: Funções em PHP

Exercício 3: Processando Dados (Com parâmetros e retorno). Você está desenvolvendo um aplicativo para pilotos de drones DJI Mini. Crie uma função chamada **calcularAutonomia(\$capacidadeBateria, \$consumoPorMinuto)**. A função deve dividir a capacidade da bateria pelo consumo para descobrir quantos minutos o drone ainda pode voar, e retornar esse valor. Ao final, guarde o retorno em uma variável e imprima na tela: "*Tempo estimado de voo: [resultado] minutos.*"

6. Lista de Exercícios: Funções em PHP

Exercício 4: Lógica e Condições dentro da Função. Vamos criar um validador de senhas simples para uma fechadura inteligente. Crie uma função chamada **verificarAcesso(\$senhaDigitada)**.

- Se a senha recebida for igual a "123456", a função deve retornar a palavra "*Acesso Liberado*".
- Caso contrário, deve retornar "*Acesso Negado*".

Teste a função duas vezes, uma com a senha correta e outra com uma senha errada, imprimindo os resultados na tela.

6. Lista de Exercícios: Funções em PHP

Exercício 5: O Grande Desafio (Funções + Arrays). Um sensor do Arduino gravou a temperatura de uma estufa durante 4 horas. Os dados estão no seguinte array:

```
$temperaturas = [22.5, 24.0, 25.5, 23.0];
```

Crie uma função chamada **calcularMediaTemperatura(\$listaDeTemperaturas)**. Esta função deve receber um array numérico, somar todos os valores dentro dele, dividir pela quantidade de itens e retornar a média final. Imprima o resultado na tela.

Dica: Vocês podem usar um laço de repetição *foreach* dentro da função para somar os valores, ou pesquisar sobre a função nativa *array_sum()* do PHP.